EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from
© Pearson Education
Hengzhao Yang
Zhifeng Zhu
Yajun Ha

上海科技大学
ShanghaiTech University

Addition & Subtraction

- Basic rules for addition

$0 + 0 = 0$	Sum of 0 with a carry of 0
$0 + 1 = 1$	Sum of 1 with a carry of 0
$1 + 0 = 1$	Sum of 1 with a carry of 0
$1 + 1 = 10$	Sum of 0 with a carry of 1

$$\begin{array}{r} 11 \\ +\ 11 \\ \hline \mathbf{110} \end{array} \qquad \begin{array}{r} 3 \\ +\ 3 \\ \hline 6 \end{array}$$

- Basic rules for subtraction

$0 - 0 = 0$	
$1 - 1 = 0$	
$1 - 0 = 1$	
$10 - 1 = 1$	$0 - 1$ with a borrow of 1

$$\begin{array}{r} 101 \\ -011 \\ \hline \mathbf{010} \end{array} \qquad \begin{array}{r} 5 \\ -3 \\ \hline 2 \end{array}$$

Binary Multiplication & Division

- Basic rules for multiplication

$$0 \times 0 = 0$$
$$0 \times 1 = 0$$
$$1 \times 0 = 0$$
$$1 \times 1 = 1$$

- Multiplication and division in binary follows the same procedure as that in decimal

```
      11          3
    × 11        × 3
    ----        ---
      11          9
Partial
products
     11
    +11
    ----
    1001
```

```
      111         7
    × 101       × 5
    -----       ---
      111        35
Partial
products
     000
    +111
    -----
   100011
```

```
      10          2
   11)110      3)6
    11          6
    ---         -
    000         0
```

```
      11          3
   10)110      2)6
    10          6
    --          -
    10          0
    10
    --
    00
```

Can we turn subtraction into addition?

Yes! motivation?

- Simplifying both hardware design and arithmetic operations (only addition hardware is needed).
- No need to determine which number is larger.

Negative Numbers and Sign-Magnitude Form

In the sign–magnitude representation, a signed number is represented by:

- the bit pattern corresponding to the sign of the number for the sign bit (often the most significant bit, set to 0 for a positive number and to 1 for a negative number),
- and the magnitude of the number (or absolute value) for the remaining bits.

Range: -127 to +127

Eight-bit sign–magnitude

Binary value	Sign–magnitude interpretation	Unsigned interpretation
00000000	0	0
00000001	1	1
⋮	⋮	⋮
01111101	125	125
01111110	126	126
01111111	127	127
10000000	−0	128
10000001	−1	129
10000010	−2	130
⋮	⋮	⋮
11111101	−125	253
11111110	−126	254
11111111	−127	255

https://en.wikipedia.org/wiki/Signed_number_representations

How to add negative numbers

- But a practical question is without using substraction, how to add negative numbers?
- Direct addition may incur mistakes:

  1+ -1 = 0

  but

  0001 + 1001=1010.

- This is solved by introducing: complement form.

1’s Complement Form

In the ones' complement representation, a negative number is represented by

- The bit pattern corresponding to the bitwise NOT (i.e. the "complement") of the positive number.
- Ones' complement has two representations of 0: 00000000 (+0) and 11111111 (−0).

$$[N] = \begin{cases} N, & N \geq 0 \\ 2^n - 1 - |N|, & N < 0 \end{cases}$$

can represent -127 to +127

Eight-bit ones' complement

Binary value	Ones' complement interpretation	Unsigned interpretation
00000000	0	0
00000001	1	1
⋮	⋮	⋮
01111101	125	125
01111110	126	126
01111111	127	127
10000000	−127	128
10000001	−126	129
10000010	−125	130
⋮	⋮	⋮
11111101	−2	253
11111110	−1	254
11111111	−0	255

https://en.wikipedia.org/wiki/Signed_number_representations

2's Complement Form

In the two's complement representation, a negative number is represented by:
the bit pattern corresponding to the bitwise NOT (i.e. the "complement") of the positive number plus one, i.e. to the ones' complement plus one.

$$N^* = 2^n - N$$

Can represent 256 numbers!
-128 to +127

Eight-bit two's complement

Binary value	Two's complement interpretation	Unsigned interpretation
00000000	0	0
00000001	1	1
⋮	⋮	⋮
01111110	126	126
01111111	127	127
10000000	−128	128
10000001	−127	129
10000010	−126	130
⋮	⋮	⋮
11111110	−2	254
11111111	−1	255

https://en.wikipedia.org/wiki/Signed_number_representations

Exercises

(1) Write down:

8 bit sign-magnitude,

1's complement form,

and 2's complement form of the following number:

- +6
- -6
- 0

(2) 16 bit 2's complement form of -6?

8-bit representations

+6:

Sign-Magnitude: 00000110

1's Complement: 00000110

2's Complement: 00000110

-6:

Sign-Magnitude: 10000110

1's Complement: 11111001

2's Complement: 11111010

0:

Sign-Magnitude: 00000000 and 10000000

1's Complement: 00000000 and 11111111

2's Complement: 00000000

16-bit representations

(2) 16 bit 2's complement form of -6?

1111 1111 1010

Compare with 8 bit 2's complement form:

11111010

2's Complement Form: Example of Adding Negative Numbers

- Both numbers positive

```
  00000111      7
+ 00000100    + 4
----------    ---
  00001011     11
```

- Positive number with magnitude larger than negative number

```
  00001111     15
+ 11111010    + −6
----------    ----
1 00001001      9   <- Discard carry
```

- Negative number with magnitude larger than positive number

```
  00010000     16
+ 11101000    + −24
----------    -----
  11111000     −8
```

- Both numbers negative

```
  11111011     −5
+ 11110111    + −9
----------    -----
1 11110010     −14   <- Discard carry
```

2's Complement Form: Example of Subtraction

- Subtracting +3 (subtrahend) from +8 (minuend) is equivalent to adding −3 to +8.
- The sign of a positive or negative binary number is changed by taking its 2's complement. (proof it)

	00001000	Minuend (+8)
	+ 11111101	2's complement of subtrahend (−3)
Discard carry ⟶	**1 00000101**	Difference (+5)

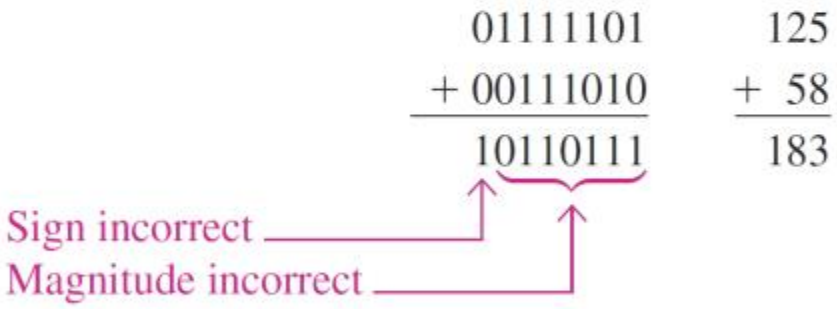
- It's important to determine the number of bits in advance

+13	0	01101	+13	0	01101
+10	0	01010	−10	1	10110
+23	0	10111	+3	0	00011
−13	1	10011	−13	1	10011
+10	0	01010	−10	1	10110
−3	1	11101	−23	1	01001

+13	0	1101	+13	0	1101
+10	0	1010	−10	1	0110
+23	1	0111	+3	0	0011
−13	1	0011	−13	1	0011
+10	0	1010	−10	1	0110
−3	1	1101	−23	0	1001

Overflow Condition

- When two numbers are added and the number of bits required to represent the sum exceeds the number of bits in the two numbers, an overflow results as indicated by an incorrect sign bit. An overflow can occur only when both numbers are positive or both numbers are negative. If the sign bit of the result is different than the sign bit of the numbers that are added, overflow is indicated.



Think about it:

- How to confidently perform the calculation like the last page?

Multiplication

Direct addition method

8	Multiplicand
× 3	Multiplier
24	Product

- Multiplication can be accomplished using addition
- Direct addition and partial products are two basic methods.
- When two binary numbers are multiplied, both numbers must be in true uncomplemented form
- In the direct addition method, one adds the multiplicand a number of times equal to the multiplier, e.g., 8 + 8 + 8 = 24.
- Multiply the signed binary numbers 01001101 and 00000100

01001101	1st time
+ 01001101	2nd time
10011010	Partial sum
+ 01001101	3rd time
11100111	Partial sum
+ 01001101	4th time
100110100	Product

Multiplication

Partial products method

Calculate in uncomplemented form, then convert back.

Step 1: Determine if the signs of the multiplicand and multiplier are the same or different. This determines what the sign of the product will be.

Step 2: Change any negative number to true (uncomplemented) form. Because most computers store negative numbers in 2's complement, a 2's complement operation is required to get the negative number into true form.

Step 3: Starting with the least significant multiplier bit, generate the partial products. When the multiplier bit is 1, the partial product is the same as the multiplicand. When the multiplier bit is 0, the partial product is zero. Shift each successive partial product one bit to the left.

Step 4: Add each successive partial product to the sum of the previous partial products to get the final product.

Step 5: If the sign bit that was determined in step 1 is negative, take the 2's complement of the product. If positive, leave the product in true form. Attach the sign bit to the product.

If negative numbers, convert to uncomplemented and then convert back.

Multiplication

Multiply the signed binary numbers: 01010011 (multiplicand) and 11000101 (multiplier).

Solution

Step 1: The sign bit of the multiplicand is 0 and the sign bit of the multiplier is 1. The sign bit of the product will be 1 (negative).

Step 2: Take the 2's complement of the multiplier to put it in true form.

$$11000101 \longrightarrow 00111011$$

Step 3 and 4: The multiplication proceeds as follows. Notice that only the magnitude bits are used in these steps.

Step 5: Since the sign of the product is a 1 as determined in step 1, take the 2's complement of the product.

$$1001100100001 \longrightarrow 0110011011111$$

Attach the sign bit →

1 0110011011111

1010011	Multiplicand
× 0111011	Multiplier
1010011	1st partial product
+ 1010011	2nd partial product
11111001	Sum of 1st and 2nd
+ 0000000	3rd partial product
011111001	Sum
+ 1010011	4th partial product
1110010001	Sum
+ 1010011	5th partial product
100011000001	Sum
+ 1010011	6th partial product
1001100100001	Sum
+ 0000000	7th partial product
1001100100001	Final product

Division

$$\frac{\text{dividend}}{\text{divisor}} = \text{quotient}$$

Brute Force division (for small numbers):

When two binary numbers are divided, both numbers must be in true (uncomplemented) form. The basic steps in a division process are as follows:

Step 1: Determine if the signs of the dividend and divisor are the same or different. This determines what the sign of the quotient will be. Initialize the quotient to zero.

Step 2: Subtract the divisor from the dividend using 2's complement addition to get the first partial remainder and add 1 to the quotient. If this partial remainder is positive, go to step 3. If the partial remainder is zero or negative, the division is complete.

Step 3: Subtract the divisor from the partial remainder and add 1 to the quotient. If the result is positive, repeat for the next partial remainder. If the result is zero or negative, the division is complete.

Division

Divide 01100100 by 00011001.

Solution

Step 1: The signs of both numbers are positive, so the quotient will be positive. The quotient is initially zero: 00000000.

Step 2: Subtract the divisor from the dividend using 2's complement addition (remember that final carries are discarded).

01100100	Dividend
+ 11100111	2's complement of divisor
01001011	Positive 1st partial remainder

Add 1 to quotient: 00000000 + 00000001 = 00000001.

Step 3: Subtract the divisor from the 1st partial remainder using 2's complement addition.

01001011	1st partial remainder
+ 11100111	2's complement of divisor
00110010	Positive 2nd partial remainder

Add 1 to quotient: 00000001 + 00000001 = 00000010.

Step 4: Subtract the divisor from the 2nd partial remainder using 2's complement addition.

00110010	2nd partial remainder
+ 11100111	2's complement of divisor
00011001	Positive 3rd partial remainder

Add 1 to quotient: 00000010 + 00000001 = 00000011.

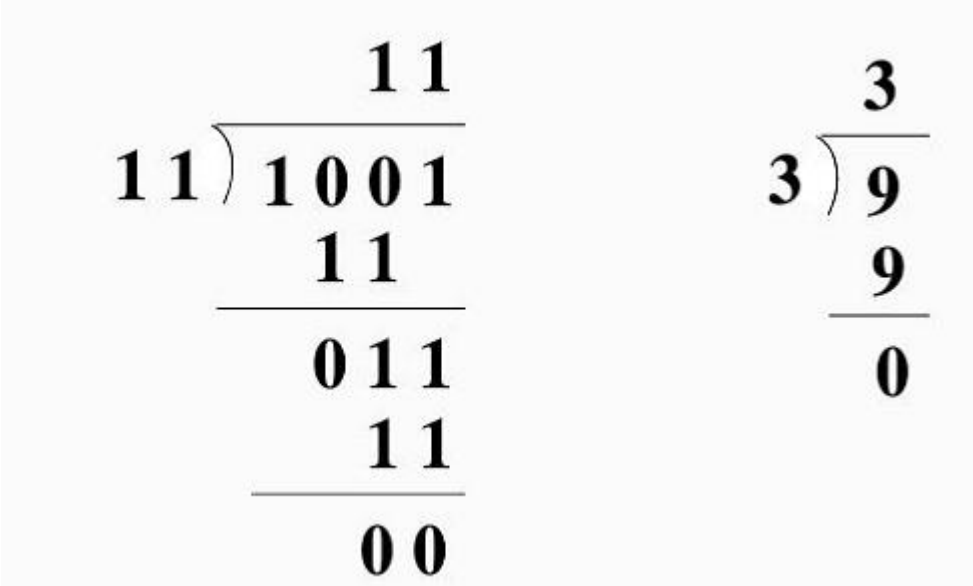
Step 5: Subtract the divisor from the 3rd partial remainder using 2's complement addition.

00011001	3rd partial remainder
+ 11100111	2's complement of divisor
00000000	Zero remainder

Add 1 to quotient: 00000011 + 00000001 = **00000100** (final quotient). The process is complete.

Division

Partial quotient method



https://fpga.eetrend.com/blog/2023/100576347.html

Algorithms for Hardware Division

- Restoring Division
- Non-Restoring Division
- SRT Division (Digit-Recurrence Division)
- ... ...

Algorithms for Hardware Multiplication

- Shift-and-Add Multiplication
- Booth Multiplication
- Modified Booth Multiplication
- ... ...